Computer Organization, Spring 2026

Homework 1:

Complete Arithmetic Logic Unit Design

Lecturer: [redacted]

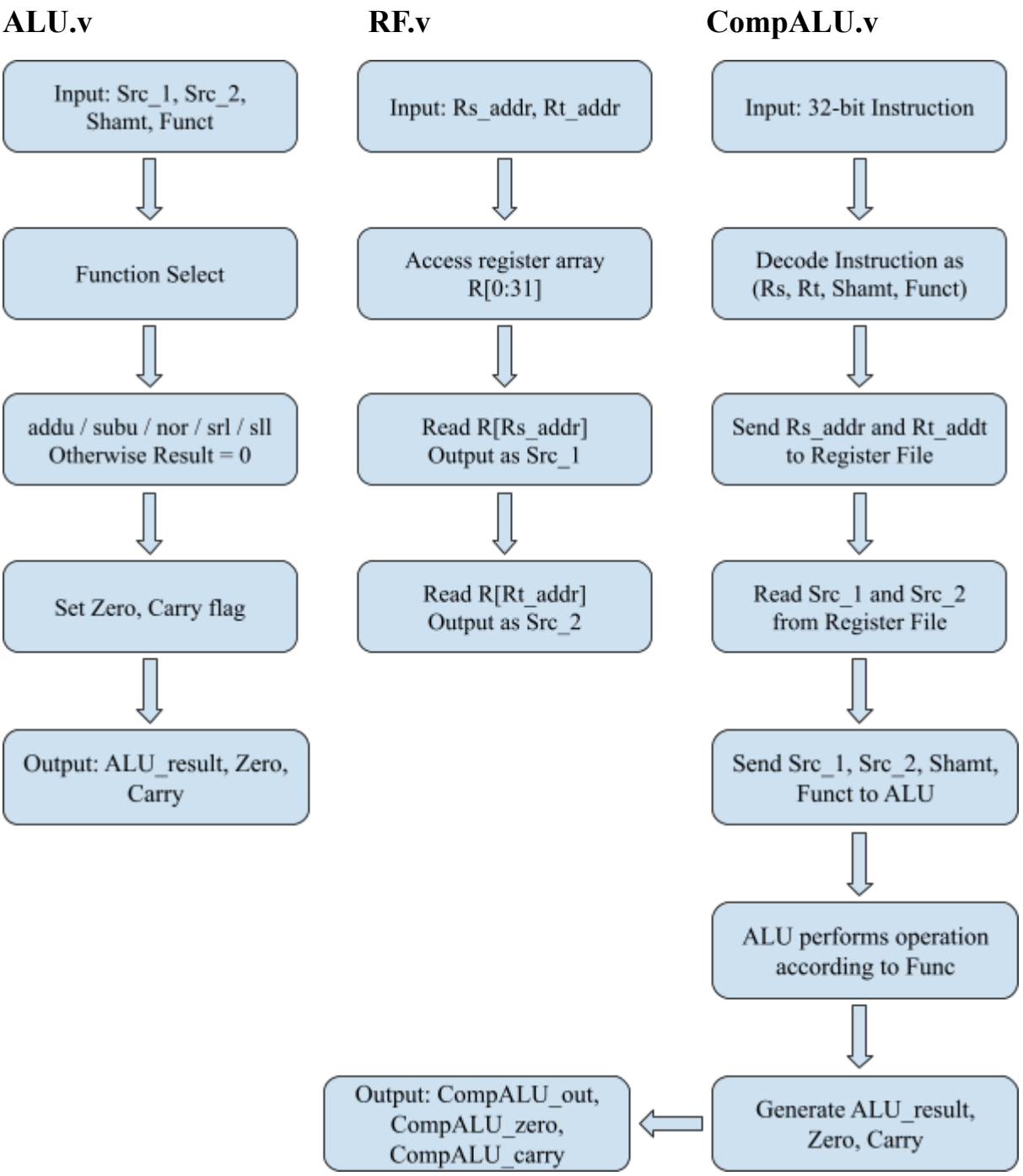
Department: Electrical Engineering

Student ID: [redacted]

Name: 胡守曾

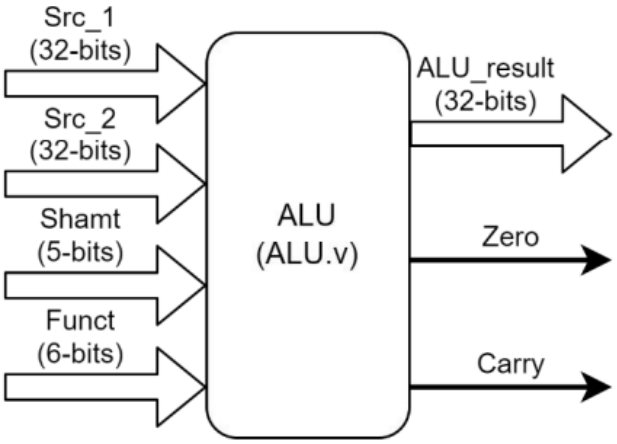
Module Description

Flow Chart

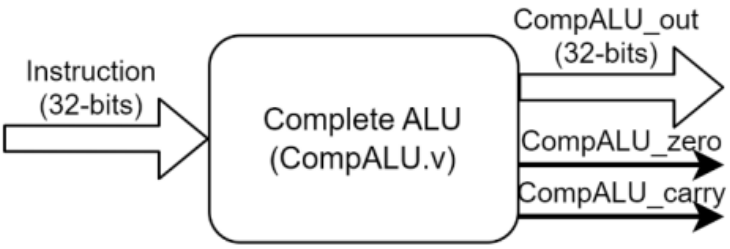


Architectures

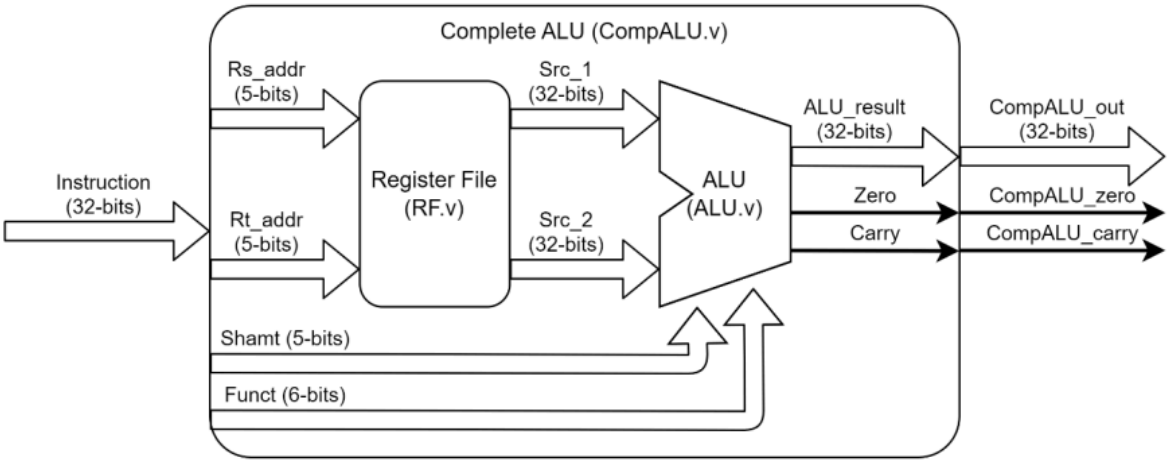
ALU.v



RF.v



CompALU.v



ALU.v

```
1   `timescale 1ns / 1ps
2   module ALU(
3    input [31:0] Src_1,
4    input [31:0] Src_2,
5    input [4:0] Shamt,
6    input [5:0] Funct,
7    output [31:0] ALU_result,
8    output Zero,
9    output Carry
10   );
11   reg [32:0] alu_out;
12
13  always@(*)begin
14      case(Funct)
15          6'b100001: begin
16              alu_out = {1'b0, Src_1} + {1'b0, Src_2};
17          end
18          6'b100011: begin
19              alu_out = {1'b0, Src_1} - {1'b0, Src_2};
20          end
21          6'b100111: alu_out = {1'b0, ~(Src_1 | Src_2)};
22          6'b000010: alu_out =  Src_2 >> Shamt;
23          6'b000000: alu_out = Src_2 << Shamt;
24          default: alu_out = 33'b0;
25          endcase
26  end
27  assign ALU_result = alu_out[31:0];
28  assign Carry = alu_out[32];
29  assign Zero = (alu_out[31:0]==32'b0);
30  endmodule
```

line	Description
2~10	I/O 介面定義
11	定義一 33 bits 內部暫存器
13~26	用case statement控制ALU做不同運算，並設有default
27~29	設定各輸出結果

Design Process

在設計ALU module時，一開始我設計了三個register，分別裝33bits輸出、32bits輸出以及1bit Carry暫存器，在case中也需要比較Src_1以及Src_2何者較大來控制carry的輸出，造成資源浪費。因此改成以下只需要一個33bits的暫存器，Carry部分則在最高位會自動產生並輸出。

RF.v

```
1   `timescale 1ns / 1ps
2   module RF(
3       input [4:0] Rs_addr,
4       input [4:0] Rt_addr,
5       output [31:0] Src_1,
6       output [31:0] Src_2
7       );
8   reg [31:0] R [0:31];
9
10  assign Src_1 = R[Rs_addr];
11  assign Src_2 = R[Rt_addr];
12
13  endmodule
```

line	Description
2~7	I/O介面定義
8	宣告暫存器資料包含32個暫存器，每一個暫存器有32bits
10	設定輸出，以輸入為地址找到資料輸出

Design Process

這個module相當簡單，因為是combinational read，因此不需要加上clk。

CompALU.v

```
1   `timescale 1ns / 1ps
2   module CompALU(
3       input [31:0] Instruction,
4       output [31:0] CompALU_out,
5       output CompALU_zero,
6       output CompALU_carry
7       );
8   wire [31:0] Src_1, Src_2;
9   wire [4:0] Rs = Instruction[25:21];
10  wire [4:0] Rt = Instruction[20:16];
11  wire [5:0] Func = Instruction[5:0];
12  wire [4:0] Shamt = Instruction[10:6];
13  RF Register_File(
14      .Rs_addr(Rs),
15      .Rt_addr(Rt),
16      .Src_1(Src_1),
17      .Src_2(Src_2)
18  );
19  ALU ALU_File(
20      .Src_1(Src_1),
21      .Src_2(Src_2),
22      .Shamt(Shamt),
23      .Funct(Func),
24      .ALU_result(CompALU_out),
25      .Zero(CompALU_zero),
26      .Carry(CompALU_carry)
27  );
28
29  endmodule
```

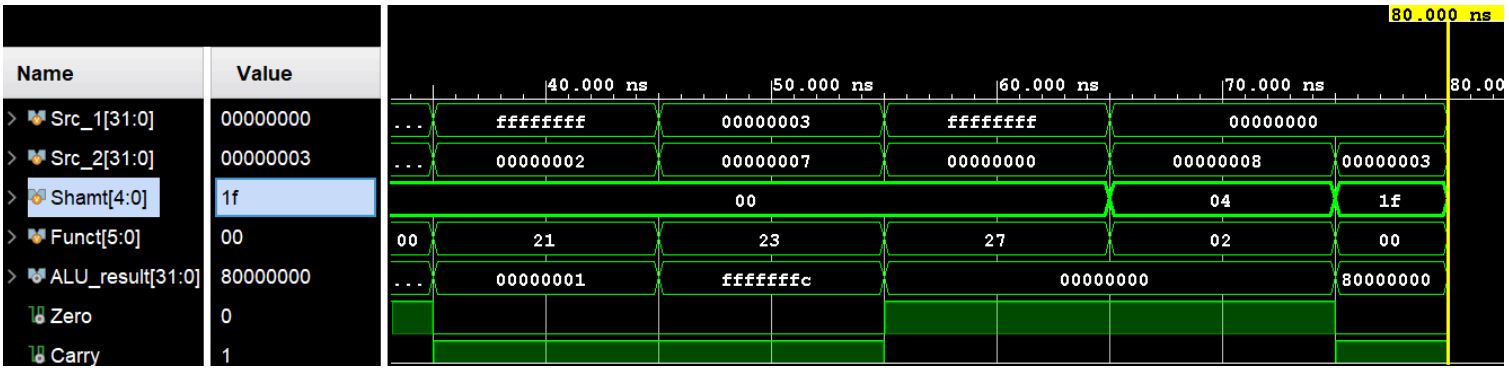
line	Description
2~7	I/O介面定義
8~12	將Instrucion中的不同位元解析，以wire連接準備做輸入
13~27	建立RF、ALU模組實例，輸入分別以剛剛設好的wire連接，先透過Register_File 來獲取資料，再透過ALU_File 來控制運算並輸出結果

Design Process

這個module在做前兩個模組的整合，內部只需要接線就好，也不需要做運算。在測試階段我額外多加了Src_1、Src_2的wire，並修正testbench來更好的判斷CompALU_out是否正確

Behavioral Simulation

tb_ALU.v



我總共設定了5組test patterns，每個Funct兩次來確認我的輸出(Zero、Carry、輸出資料)是否正確。

五組分別為addu、subu、nor、right shift、left shift

1. ffff-ffff + 0000-0002 (addu)

   ALU_result = 0000-0001, Zero = 0, Carry = 1

2. 0000-0003 - 0000-0007 (subu)

   ALU_result = ffff-fffc, Zero = 0, Carry = 1

   # Src_1 < Src_2, Carry = 1

3. ~(ffff-ffff | 0000-0000) (nor)

   ALU_result = 0000-0000, Zero = 1, Carry = 0

4. 0000-0008 >> 04 (right shift)

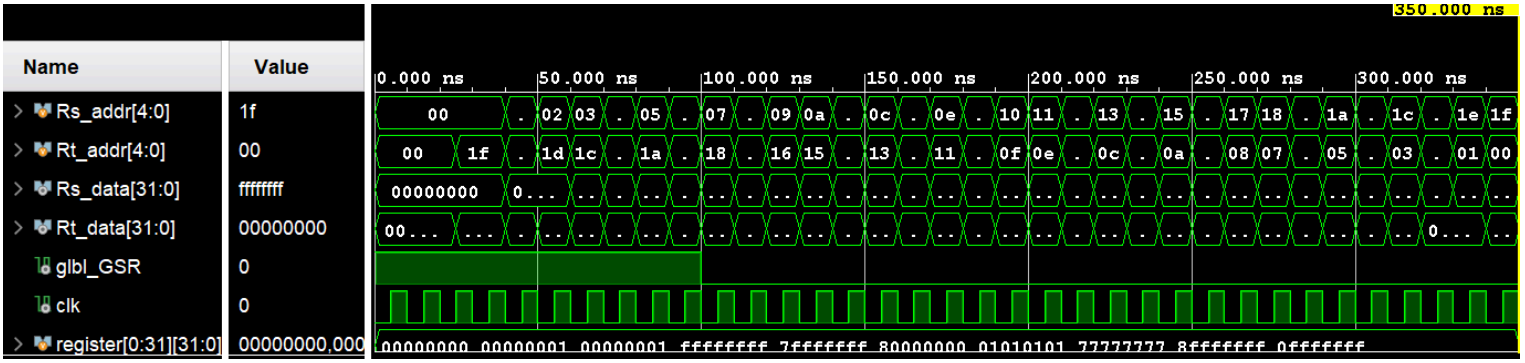
   ALU_result = 0000-0000, Zero = 1, Carry = 0

5. 0000-0003 << 1f (left shift)

   ALU_result = 8000-0000, Zero = 0, Carry = 1

   # 向左位移31bit，造成溢位，Carry = 1

tb_RF.v

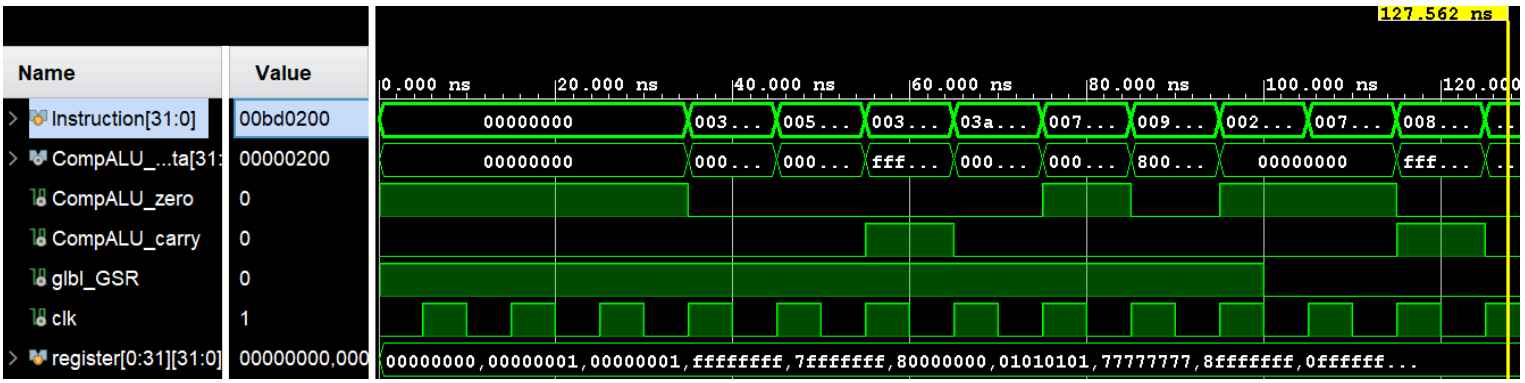


從結果中可以看到透過clk 正緣觸發輸出資料，test pattern設計可以讓我們發現資料都一樣，輸入地址則為相反的，5bits地址總共可以有32bits值。

```
0000_0000
0000_0001
0000_0001
FFFF_FFFF
7FFF_FFFF
8000_0000
0101_0101
7777_7777
8FFF_FFFF
0FFF_FFFF
1234_1234
4321_4321
1234_5678
8765_4321
8080_8080
8888_8888
7F7F_7F7F
F7F7_F7F7
ABCD_DCBA
0123_4567
89AB_CDEF
0246_8ACE
1357_9BDF
0000_0018
0000_0019
0000_001A
0000_2000
0000_0200
0000_0020
0000_0002
7F7F_7F7F
FFFF_FFFF
```

此為 Register File 的內部儲存資料。透過 Rs_addr 與 Rt_addr 作為索引，分別輸出 Src_1 與 Src_2。其中 Src_1 依位址由上至下選取，而Src_2則為反向選取機制。

tb_CompALU.v



Instrucions:

我總共設了10筆測試資料, 以此判斷功能是否正確, 以下是共十組測試資料:

1. 000000 00001 11101 00000 11111 100001 (addu)

   # 找到地址00001的值並與地址11101的值相加, 透過RF資料可知道為

   0000-0001 + 0000-0002 = 0000-0003

   Carry: 0 Zero: 0

2. 000000 00010 11100 00000 00000 100001 (addu)

   result: 0000-0001 + 0000-0020 = 0000-0021

   Carry: 0 Zero: 0

3. 000000 00001 11101 00000 11111 100011 (subu)

   result: 0000-0001 - 0000-0002 = ffff-ffff

   Carry: 1 Zero: 0 #Src_1 < Src_2 因此carry = 1

4. 000000 11101 00001 00000 00000 100011 (subu)

   result: 0000-0002 - 0000-0001 = 0000-0001

   Carry: 0 Zero: 0

5. 000000 00011 11100 00000 11111 100111 (nor)

   result: ~(ffff-ffff | 0000-0020) = 0000-0000

   Carry: 0 Zero: 1

6. 000000 00100 11011 00000 00000 100111 (nor)

   result: ~(7fff-ffff | 0000-0200) =8000-0000

   Carry: 0 Zero: 0

7. 000000 00001 00010 00000 00001 000010 (right shift)
   result: 0000-0001 >> 0001 = 0000-0000
   Carry: 0 Zero: 1
8. 000000 00011 11001 00000 01000 000010 (right shift)
   result: 0000-001a>> 01000 = 0000-0000
   Carry: 0 Zero: 0
9. 000000 00100 00011 00000 00001 000000 (left shift)
   result: ffff-ffff << 00001 = ffff-fffe
   Carry: 1 Zero: 0
10. 000000 00101 11101 00000 01000 000000 (left shift)
   result: 0000-0002 << 01000 = 0000-0200
   Carry: 0 Zero: 0

Conclusion

在這次作業中，我完成了一個32bits的ALU系統設計，並用Verilog HDL進行實作與模擬驗證，整個架構可以分為三個部分，Register File、ALU、以及整合二者的CompALU，透過simulation可以確認每個模組皆能正常運作，同時設計不同test patterns 來判斷整個程式碼是否達到目的。

透過這次作業，我更深入理解了數位電路設計與硬體描述語言的基礎觀念。雖然這次作業基本上是Combination Logic，因此不需要使用clk來控制，也不需要FSM來操作流程，但這樣的練習依然讓我學到了許多。尤其是優化後透過33bits暫存器來處理加減法等運算，讓我理解到了硬體設計中是如何保留carry或borrow bit的。

目前ALU僅能操作五種功能，在未來可以進一步擴充更多邏輯與算術功能。總體而言，這次作業讓我實際體驗了從模組設計、系統整合到模擬驗證的完整流程。